Artificial Intelligence

BS (CS) Spring 2026

Lab 09 Tasks

Learning Objectives:

1. Adversarial Search

Lab Tasks

Submission Instructions

1. Create one notebook file for all questions with name *ROLLNO_SEC_LAB09* **e.g. i23XXXX_A_LAB09**
2. Submit your .ipynb file with correct name on Google Classroom under your section heading.
3. If you don't follow the above-mentioned submission instruction, you will be marked **zero.**

Plagiarism or AI in the Lab Task will result in **zero** marks in the whole category.

Question

Two AI-controlled railway companies compete over a limited set of high-value routes on a simplified railway network. Each route connects two cities and can be claimed by only one player. The objective for each company is to **maximize its total score** by claiming strategic routes while **preventing the opponent from forming long, high-scoring connections**.

This problem presents a **two-player adversarial environment** where decisions must consider both immediate gains and future consequences. The AI (MAX player) will use the **Minimax algorithm with depth-limited search** to select the optimal moves, anticipating the opponent's (MIN player) responses.

Objective:

Design and implement a **Minimax-based decision system** for a two-player adversarial environment.

Requirements:

- **Game Representation**

  - Represent the railway network as a **graph using adjacency lists**.
  - Each node represents a city; each edge represents a route with associated points.
  - Track which routes have been claimed and by which player.

Example (Map Adjacency List):

Route	Status	Score if claimed
A-B	Unclaimed	5
A-C	Unclaimed	3
A-D	Unclaimed	4
B-C	Unclaimed	2
B-D	Unclaimed	6
C-D	Unclaimed	2

C-E	Unclaimed	5
D-E	Unclaimed	6

```
graph = {
    "A": [{"to": "B", "points": 5, "claimed": False},
          {"to": "C", "points": 3, "claimed": False},
          {"to": "D", "points": 4, "claimed": False}],
    "B": [{"to": "A", "points": 5, "claimed": False},
          {"to": "C", "points": 2, "claimed": False},
          {"to": "D", "points": 6, "claimed": False}],
    "C": [{"to": "A", "points": 3, "claimed": False},
          {"to": "B", "points": 2, "claimed": False},
          {"to": "D", "points": 2, "claimed": False},
          {"to": "E", "points": 5, "claimed": False}],
    "D": [{"to": "A", "points": 4, "claimed": False},
          {"to": "B", "points": 6, "claimed": False},
          {"to": "C", "points": 2, "claimed": False},
          {"to": "E", "points": 6, "claimed": False}],
    "E": [{"to": "C", "points": 5, "claimed": False},
          {"to": "D", "points": 6, "claimed": False}]
}
```

- **Game State**

Each state must include:

- Claimed and unclaimed routes.
- Current scores of both players.

- **Players**

- **MAX player**: The AI making optimal decisions.
- **MIN player**: The opponent, assumed to play optimally.

- **Decision-Making**

- Implement a **depth-limited Minimax algorithm** (depth = 3 or 4) to select moves.
- Generate all legal moves, apply a move to the state, and recursively evaluate outcomes.

- **Evaluation Function**

Define a heuristic utility function to evaluate non-terminal states:

- Reward points for claimed routes.
- Bonus for completing connections between cities.
- Penalty if the opponent blocks critical routes.

Tasks:

1. Define a structured game state representation.

2. Implement `MOVE_GEN(state)` to generate all legal route claims.
3. Implement `APPLY_MOVE(state, move, player)` to update the state.
4. Define a **utility function**:
    - +route points
    - +bonus for connecting cities
    - −penalty if opponent blocks key routes
5. Implement recursive Minimax.
6. Demonstrate how the AI selects the optimal move.

Starter Code:

```
class GameState:
    def __init__(self, routes, scores):
        self.routes = routes
        self.scores = scores

def generate_moves(state):
    return [r for r in state.routes if not r['claimed']]

def apply_move(state, move, player):
    new_state = deepcopy(state)
    move['claimed'] = True
    new_state.scores[player] += move['points']
    return new_state

def evaluate(state):
    return state.scores["MAX"] - state.scores["MIN"]
```